R3.04 Qualité de développement : bonnes pratiques

SOLID et quelques-uns de ses amis les acronymes

Avant SOLID : ce que vous savez déjà...

Les règles de codage / bonnes pratiques / conventions / normes de programmation sont un ensemble de règles visant à uniformiser le code source d’un projet.

Trop souvent oubliées par les développeurs débutants.

Buts :

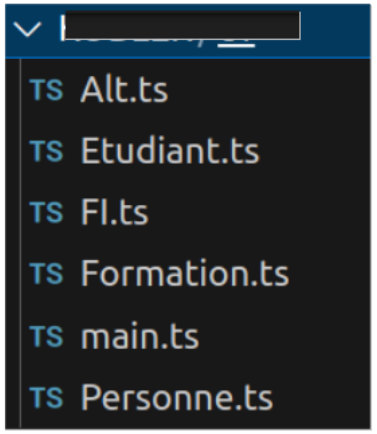
- meilleure lisibilité du code : c’est encore plus vrai si vous travaillez en équipe ; il faut écrire du code propre pour que vos collègues puissent vous relire.
- meilleure maintenabilité : un code plus lisible et mieux commenté est bien plus facile à modifier (correction de bugs, ajout de nouvelles fonctionnalités)
- → gain de temps et d’énergie.

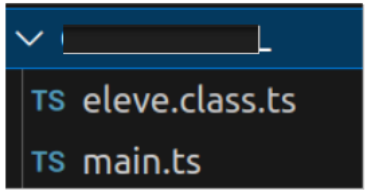
Règle 1 : nommer correctement les fichiers

Normes souvent dépendantes des conventions définies dans les entreprises / équipes.

Principe : nommer ses fichiers en fonction de ce qui y est défini : classe `Automobile` dans `Automobile.ts`. Sous-entendu : une classe par fichier !

A gauche : j'ouvre le projet, je vois tout de suite l'architecture du code (même si certains noms sont abrégés donc peu clairs)





A droite : je dois parcourir le fichier `eleve.class.ts` pour y chercher les différentes classes du projet (ET mon sujet concerne des étudiants, pas des élèves !)

Règle 2 : nommage des classes, variables, fonctions, etc.

Ces règles peuvent changer d’une entreprise/équipe à l’autre.

Mais : des régles communes à tout le monde.

Le nom des éléments doit avoir du sens. En lisant l'identificateur, on comprend immédiatement à quoi on sera confronté.

```
let etudiant : Candidat; // oui
let myObject : Candidat; // NON
```

Exemple de règles "Bonnes pratiques pour l'utilisation d'AWS CDK dans TypeScript pour créer des projets IaC" :

Guidance > Bonnes pratiques pour l'utilisation d'AWS CDK dans TypeScript pour créer des projets IaC

Définition des conventions de dénomination standard

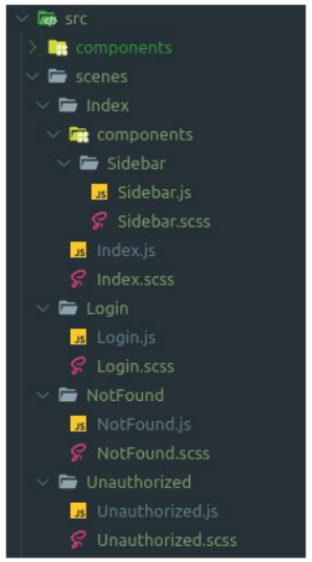
L'application d'une convention de dénomination permet de maintenir la cohérence de la base de code et de réduire la surcharge lorsque vous réfléchissez à la manière de nommer une variable. Nous vous recommandons la procédure suivante :

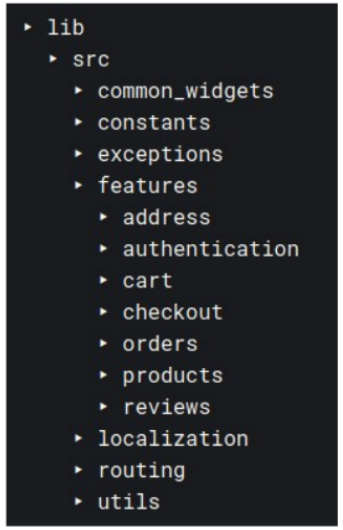
- Utilisez CamelCase pour les noms de variables et de fonctions.
- Utilisez PascalCase pour les noms de classes et d'interfaces.
- Utilisez CamelCase pour les membres de l'interface.
- Utilisez PascalCase pour les noms de types et les noms d'énumérations.
- Nommez les fichiers avec CamelCase (par exemple, `ebsVolumes.tsx` ou `storage.tsb`).

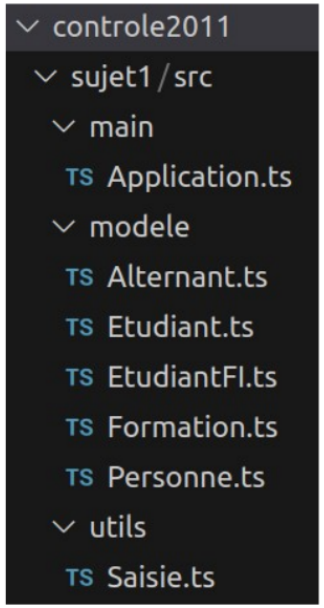
Règle 3 : architecture du projet

- Métaphore de la maison / pièces / meubles / tiroirs

```
your_project/
├─ assets/
├─ bin/
│  └─ console
├─ config/
│  ├─ packages/
│  └─ services.yaml
├─ migrations/
├─ public/
│  ├─ build/
│  └─ index.php
├─ src/
│  ├─ Kernel.php
│  ├─ Command/
│  ├─ Controller/
│  ├─ DataFixtures/
│  ├─ Entity/
│  ├─ EventSubscriber/
│  ├─ Form/
│  ├─ Repository/
│  ├─ Security/
│  └─ Twig/
├─ templates/
├─ tests/
├─ translations/
├─ var/
│  ├─ cache/
│  └─ log/
└─ vendor/
```







Exemples d'architecture de projets Symfony / React.js / Flutter / TS

Règle 4 : taille des dossiers, fichiers, classes, fonctions, etc.

- Personne n'aime :
  - ouvrir un fichier de 1500 lignes
  - lire une fonction de 300 lignes.
- On s'y perd
- Pour comprendre ce que fait la fonction, il faut lire 300 lignes !
- Solution : décomposer en sous-fonctions, plus lisibles et réutilisables
- Norme possible : une fonction doit être entièrement visible sans scroller.
- Une classe trop longue fait sans doute trop de choses différentes -> à décomposer en plusieurs classes mieux réutilisables !

Règle 5 : commentaires

- Ne pas tomber dans l’excès et commenter chaque ligne de code ;
- Mettre des commentaires pour chaque classe, fonctions, et blocs **un peu complexes**.
- Si les règles 1 à 4 sont bien respectées, l'ajout de commentaires peut être limité !
- Remarque : génération de documentation à partir des commentaires du code

TSDoc

Intro

Choose a sample... | Light Theme

```
/**
 * Returns the average of two numbers.
 *
 * @remarks
 * This method is part of the {@link core-library#Statistics | Statistics subsystem}.
 *
 * @param x - The first input number
 * @param y - The second input number
 * @returns The arithmetic mean of `x` and `y`
 *
 * @beta
 */
function getAverage(x: number, y: number): number {
  return (x + y) / 2.0;
}

// The TypeScript function is included for illustrative purposes.
// It is not processed by the TSDoc parser.
```

HTML | DOM | Lines | AST | Emitter

Summary

Returns the average of two numbers.

Parameters

Name	Description
x	The first input number
y	The second input number

Return Value

Règle 6 : indentation du code

Vous connaissez tous cela...

Avoir un code bien indenté est essentiel à sa bonne lecture et maintenabilité.

Mais :

- certains utilisent les tabulations ;
- d’autres des espaces
  - deux espaces ;
  - quatre espaces, etc.

Ces règles dépendent de chaque développeur, équipe ou entreprise.

A définir avec un bon IDE.

SOLID

- Acronyme qui regroupe cinq principes de programmation
- But : produire des architectures logicielles plus compréhensibles, flexibles et maintenables
- Origine : Robert C. Martin, *Design Principles and Design Patterns*, 2000

S : Single Responsability Principle (1)

- **Principe de Responsabilité Unique**
  - une classe n’a qu’une seule raison de changer
  - une classe ne remplit qu’un seul rôle
  - une classe n’a qu’une seule responsabilité
- Exemple de rôles :
  - Représentation d'une entité
  - Affichage
  - Sauvegarde en base
  - Envoi d'emails

S : Single Responsability Principle (2)

- Ce qu'il faut vérifier / Ce qu'on veut obtenir
  - Une classe ne stocke pas plusieurs groupes de données différentes
  - Les fonctionnalités de chaque classe sont le plus réduites possibles
  - Beaucoup de classes, peu de méthodes dans chaque classe
- Exemple de clases qui ont plusieurs responsabilités

```
export class Eleve_ALT extends Eleve {
    private _nom_ent! : string;
    private _nom_mstage! : string;
    private _prenom_mstage! : string;
```

```
export class Etudiant {
    protected _nom: string;
    protected _prenom: string;
    protected _INE: string;
    protected _formation: string;
    protected _annee_debut: string;
    protected _annee_fin: string;
```

(un alternant ET son maître de stage ; un étudiant ET sa formation)

Parenthèse : SRP $\Longrightarrow$ MVC (1)

- MVC = Modèle, Vue, Contrôleur
  - Modèle : on représente les données
  - Vue : on affiche les données
  - Contrôleur : on réagit aux événements, qui modifient les données, ce qui nécessite de prévenir la vue qu'elle doit se mettre à jour

```
class Etudiant {
  private nom: string;private prenom: string;private ine: string;

  saisirNom(): void {
    let saisie: string;
    do { saisie = prompt("Veuillez saisir le nom") || "";
    } while (saisie.trim() === "");
    this.setNom(saisie.trim());
  }
  afficherInformation(): void {
    console.log(`${this.prenom} ${this.nom} (${this.ine})`);
  }
```

Parenthèse : SRP $\Longrightarrow$ MVC (2)

- affiche dans la console, et c'est tout ! Que faire si je veux utiliser cette classe dans une app mobile, ou *desktop*, hors console ?
- Erreur faite → cette classe a trois responsabilités : le stockage des données (M), la saisie (V, C) et l'affichage (V)
- Version correcte :

```
class Etudiant {
  ...
  public toString() : string {
    return `${this.prenom} ${this.nom} (${this.ine})`
  }
}
// Et la saisie se fait ailleurs (main, menu, etc.)
```

- donne une version "chaîne" de l'étudiant, utilisable sur une application *desktop*, en html, etc. Version beaucoup plus réutilisable, on reste dans la couche Modèle

O : Open-Closed Principle (1)

- Principe Ouvert-Fermé
- Une classe doit :
  - être ouverte à l'extension
  - être fermée à la modification
- Idée : une classe qui fonctionne depuis longtemps ne doit pas être modifiée
  - nécessité de la tester à nouveau après modification
  - risque d'effets de bord indésirables
- Conséquence :
  - On ne modifie que pour corriger des bugs
  - Pour ajouter des fonctionnalités, on utilise
    - l'héritage
    - la surcharge de méthode
    - des *design patterns* de structure (nous en reparlerons)

O : Open-Closed Principle (2)

```
class Etudiant extends Personne {
    private _ine! : string;
    private _formation : Formation;
    private _statut! : string;
    private _titreProjet! : string;
    private _nomEntreprise! : string;
    private _maitre : Personne;

    public toString() : string {
      let ch = super.toString() + "(" + this._ine + "), étudiant en " + this._formation.toString();

      if (this._statut.toLowerCase() === "fi"){
          ch += ", étudiant de formation initiale travaillant sur le projet " + this._titreProjet;
      } else if (this._statut.toLowerCase() === "alt"){
          ch += ", alternant chez " + this._nomEntreprise + ", maître de stage " + this._maitre.toString();
      }
      return ch;
    }
```

Problème : si on ajoute un nouveau statut d'étudiant (exemple : formation continue)

- il faut ajouter de nouveaux attributs et les méthodes correspondantes
- il faut modifier la méthode `toString` en ajoutant un `else if`
- retester **toute la classe** avant de la mettre en production

O : Open-Closed Principle (3)

```
export abstract class Etudiant extends Personne {
  private _ine!: string; private _formation: Formation;

  public toString(): string {
    return `${super.toString()} (${this.ine}), étudiant en ${this.formation.toString()}`;
  }}
export class EtudiantFI extends Etudiant {
  private _titreProjetTutore!: string;

  public toString(): string {
    return `${super.toString()}, étudiant de formation initiale travaillant sur le projet ${this.titreProjetTutore}`;
  }}
export class Alternant extends Etudiant {
  private _entreprise!: string; private _maitreDeStage!: Personne;

  public toString(): string {
    return `${super.toString()}, alternant chez ${this.entreprise}, maître de stage ${this.maitreDeStage.toString()}`;
  }}
```

Ajout d'un étudiant de formation continue :

- ajout d'une classe `EtudiantFC` avec ses attributs et méthodes propres, et une surcharge de la méthode `toString()`
- les trois autres classes ne sont pas modifiées
- on ne doit tester que la nouvelle classe `EtudiantFC` avant la mise en production.

L : Liskov Substitution Principle (1)

- Principe de substitution de Liskov
- Si q(x) est une propriété démontrable pour tout objet x de type T, alors q(y) est vraie pour tout objet y de type S tel que S est un sous-type de T.
- Si « S » est un sous-type de « T », alors tout objet de type « T » peut être remplacé par un objet de type « S » sans altérer les propriétés désirables du programme concerné.
- Exemple : dans la "vraie vie", un carré est une sorte de rectangle
- Donc `Carré` hérite de `Rectangle` ?

L : Liskov Substitution Principle (2)

```
class Rectangle {

    private _longueur : number;
    private _largeur : number;

    constructor(l1:number, l2:number) {
        this.largeur = l1; this.longueur = l2;
    }
    public set largeur(l : number) {              public set longueur(l : number) {
        this._largeur = l;                          this._longueur = l;
    }                                             }
}

class Carre extends Rectangle {

    constructor(c:number) {
        super(c, c);
    }
}

let r = new Rectangle(10, 20);
let c = new Carre(5);
console.log(r, c); // Rectangle { _largeur: 10, _longueur: 20 } Carre { _largeur: 5, _longueur: 5 }
```

L : Liskov Substitution Principle (3)

- Selon le principe de Liskov, toutes les méthodes de `Rectangle` doivent être utilisables avec `Carré`

```
c.longueur = 10;
console.log(c); // Carre { _largeur: 5, _longueur: 10 }
```

- "Solution" (bricolage infâme) :

```
class Carre extends Rectangle {

    constructor(c:number) {
        super(c, c);
    }

    public set largeur(l:number) {
        super.largeur=l;
        super.longueur=l;
    }
}
c.longueur = 10;
console.log(c); // Carre { _largeur: 10, _longueur: 10 }
```

```
public set longueur(l:number) {
    super.largeur=l;
    super.longueur=l;
}
```

L : Liskov Substitution Principle (4)

- Conclusion : si un carré est une sorte de rectangle dans la "vraie vie", ce n'est pas le cas en modélisation objets !

I : Interface Segregation Principle (1)

- Ségrégation des interfaces : préférer plusieurs interfaces avec peu de méthodes qu'une seule interface avec beaucoup de méthodes (SRP des interfaces ?)
- Rappel : une interface est une classe abstraite qui ne contient que des méthodes abstraites
- (En TypeScript, il peut y avoir des attributs dans une interface. Passons...)

I : Interface Segregation Principle (2)

- Interface, ou Classe abstraite ?
- Exemple: gestion de figures

```
abstract class Figure {

  abstract calculAire() : number;
}

class Rectangle extends Figure {

  private _longueur : number;
  private _largeur : number;

  calculAire() : number {
    return this._largeur * this._longueur;
  }
}
```

```
class Triangle extends Figure {

    private _base : number;
    private _hauteur : number;

    calculAire() : number {
      return this._base * this._hauteur / 2;
    }
}
```

- Toutes les figures (et seules les figures) doivent calculer leur aire, une classe abstraite suffit...

I : Interface Segregation Principle (2)

- Interface, ou Classe abstraite ? (suite)
- Evolution du logiciel : un dessin, qui intègre toutes les figures, doit pouvoir calculer l'aire totale de ses figures.
- Un dessin contient une liste de figures, ce n'est pas une sorte de figure, donc `Dessin` n'hérite pas de `Figure`.
- Donc `calculAire()` ne peut plus être défini dans `Figure`
- La solution est de créer une interface partagée (implémentée !) par `Figure` et `Dessin`

I : Interface Segregation Principle (3)

```
export interface ICalculateurAire {
    calculAire() : number;
}

abstract class Figure implements ICalculateurAire {
    abstract calculAire() : number;
}

class Rectangle extends Figure implements ICalculateurAire { ... }
class Triangle extends Figure implements ICalculateurAire { ... }

class Dessin implements ICalculateurAire {

    private _figures : Array<Figure>;

    calculAire() : number {
        let aireTotale = 0;
        for (let figure of this._figures) {
            aireTotale += figure.calculAire();
        }
        return aireTotale;
    }
}
```

I : Interface Segregation Principle (4)

- le principe de ségragation des interfaces va privilégier l'utilisation d'interfaces possédant peu de méthodes
- imaginons qu'on souhaite décaler les figures d'un certain nombre de pixels, mais qu'on ne le fasse pas sur le dessin, et qu'on a écrit cette interface :

```
interface IOperationsFigures {
    calculAire() : number;
    decale(x : number, y : number) : void;
}
```

- On va devoir faire :

```
class Dessin implements IOperationsFigures {
    private _figures : Array<Figure>;

    decale(x:number, y:number) {}    // Implémentation vide obligatoire !
    calculAire() : number { ... }
}
```

I : Interface Segregation Principle (5)

- Il aurait fallu faire :

```
interface ICalculateurAire {
    calculAire() : number;
}

interface IDecalage {
    decale(x : number, y : number) : void;
}

class Rectangle extends Figure implements ICalculateurAire, IDecalage { ... }
class Triangle extends Figure implements ICalculateurAire, IDecalage { ... }

class Dessin implements ICalculateurAire { ... }
```

D : Dependency Inversion Principle (1)

- Une classe doit dépendre des abstractions, pas des implémentations
- Le but est d'obtenir un code plus facilement réutilisable
- Dans le code suivant, la classe `FigureDAO` ne peut être utilisée qu'avec une instance de la classe `MySQLConnector`

```
class FigureDAO {

  private _connecteur : MySQLConnector;

  public creeConnexion() {
    this._connecteur.connexion();
  }
}
class MySQLConnector {

  public connexion() {
    ...
  }
}
```

D : Dependency Inversion Principle (2)

- Si on veut modifier la solution de persistance, pour se connecter, par exemple, à Oracle, on doit modifier la classe
- L'utiisation d'une interface aurait pu éviter cela

```
interface Iconnector {
  connexion();
}
class MySQLConnector implements IConnector {
  public connexion() {...}
}
class OracleConnector implements IConnector {
  public connexion() {...}
}
class FigureDAO {
  private _connecteur : IConnector;

  public creeConnexion() {
    this._connecteur.connexion();
  }
}
```

SOLID : conclusion

- SOLID donne des principes qui permettent de développer du code facilement modifiable, réutilisable, facile à lire, bien architecturé
- L'idée est d'avoir ces principes en tête lorsqu'on programme
- Après avoir écrit du code, on peut l'analyser : est-il SOLID ? Et éventuellement effectuer du *refactoring* pour corriger les faiblesses
- Tout ce qui concerne les interfaces est un peu plus complexe...

En guise de conclusion...

Quelques autres acronymes :

YAGNI : You Ain't Gonna Need It

KISS : Keep It Simple, Stupid!

DRY : Don't Repeat Yourself